

Experiment : 02

Aim: Write a python program to implement water Jug Problem.

Theory: The Water Jug Problem is a classic example in Artificial Intelligence that demonstrates how problems can be modelled and solved using state-space search. It involves two jugs with known capacities and an unlimited supply of water. The objective is to measure a specific quantity of water (e.g., 2 liters) using only these two jugs and a limited set of operations such as filling, emptying, or transferring water between the jugs. Each unique state of water in the jugs can be represented as a node in a graph, and transitions between states are driven by the permitted operations. This problem helps illustrate fundamental AI concepts like goal-driven search, problem decomposition, and the use of algorithms like Breadth-First Search (BFS) or DepthFirst Search (DFS) to explore the solution space. Though theoretical, it serves as a foundation for solving real-world problems involving planning, constraint satisfaction, and automated decision-making. The problem presents two jugs with fixed capacities and asks for a method to measure an exact amount of water using the jugs and permitted operations:

Algorithm:

1. Start with two empty jugs (initial state).
2. Enqueue the initial state in a queue.
3. While the queue is not empty:
 - a. Dequeue a state (a, b).
 - b. If a or b equals the target, return success.

- c. Generate all possible states by applying:
 - Fill Jug1
 - Fill Jug2
 - Empty Jug1
 - Empty Jug2
 - Pour Jug1 → Jug2
 - Pour Jug2 → Jug1
- d. Enqueue all unvisited valid states.
- 4. If all states are explored and target not found, return failure.

Program:

```
from collections import deque
import time
def water_jug_bfs(jug1,jug2,target):
    start_time=time.time()
    visited=set()
    queue=deque()
    queue.append((0,0))
    while queue:
        a,b=queue.popleft()
        if (a,b) in visited:
            continue
        visited.add((a,b))
        if a == target or b == target:
            print("solution found:",(a,b))
            print("step_Taken:",len(visited))
            print("time taken:",time.time()-start_time)
            return
        queue.extend([
```

```
(jug1,b),(a,jug2),(0,b),(a,0),  
(a-min(a,jug2-b),b+min(a,jug2-b)),  
(a+min(b,jug1-a),b-min(b,jug1-a))  
])
```

water_jug_bfs(4,3,2)

Output: